

JAVASCRIPT

1. Introduction to JavaScript

- **What is JavaScript?**
 - History and evolution
 - Client-side vs. server-side scripting
- **Setting up your environment**
 - Browsers and Developer Tools
 - Installing a code editor (e.g., VS Code)
 - Basic debugging in Developer Tools

2. JavaScript Basics

- **Syntax and Fundamentals**
 - Variables (`var`, `let`, `const`)
 - Data types (string, number, boolean, null, undefined, symbol, bigint)
 - Operators (arithmetic, assignment, comparison, logical)
- **Control Structures**
 - Conditionals (`if`, `else`, `else if`, `switch`)
 - Loops (`for`, `while`, `do...while`, `for...of`, `for...in`)
- **Functions**
 - Function declarations and expressions
 - Arrow functions
 - Parameters and return values
 - Scope and closures

3. DOM Manipulation

- **Document Object Model (DOM)**
 - Selecting elements (`getElementById`, `querySelector`, etc.)
 - Modifying HTML and CSS (`innerHTML`, `style`, `classList`)
 - Event handling (`addEventListener`, event types)
- **Forms and Input**
 - Form validation
 - Event listeners for forms
 - Dynamic form inputs

4. Intermediate Concepts

- **Arrays and Objects**
 - Creating and manipulating arrays
 - Array methods (`map`, `filter`, `reduce`, `forEach`)
 - Working with objects and object methods
- **Error Handling**
 - `try...catch`
 - `throw` statements
 - Custom error messages
- **Asynchronous JavaScript**
 - Callbacks
 - Promises (`then`, `catch`, `finally`)
 - `async/await`

5. Advanced Topics

- **ES6+ Features**
 - Destructuring
 - Template literals
 - Default parameters
 - Rest and spread operators
 - Modules (`import` and `export`)
- **Object-Oriented Programming (OOP)**
 - Prototypes and inheritance
 - Classes and constructors
 - Encapsulation, inheritance, and polymorphism
- **Functional Programming**
 - Pure functions
 - Higher-order functions
 - Immutability
 - Currying and partial application

6. Working with APIs

- **AJAX and Fetch API**
 - Making HTTP requests
 - Consuming APIs
 - Error handling in API calls
- **JSON**
 - Parsing and stringifying JSON data
 - Working with API responses

7. Browser-Specific Features

- **LocalStorage, SessionStorage, and Cookies**
- **Geolocation API**
- **Web Workers**
- **Canvas and SVG**

8. Testing and Debugging

- **Debugging Techniques**
 - Using breakpoints
 - The `console` object (`log`, `warn`, `error`, `table`)
- **Testing Frameworks**
 - Unit testing with Jest or Mocha
 - Writing test cases

9. JavaScript Build Tools

- **Introduction to npm and Yarn**
- **Module bundlers**
 - Webpack
 - Parcel
- **Transpilers**
 - Babel

10. Frameworks and Libraries

- **Introduction to Popular Frameworks**
 - React.js
 - Vue.js
 - Angular
- **Utility Libraries**
 - Lodash
 - Moment.js (or alternatives like Day.js)

11. Real-World Applications

- **Building Projects**
 - To-Do List
 - Weather App (using an API)
 - Portfolio Website
 - E-commerce Cart
 - Real-time Chat App
- **Version Control**
 - Git basics

- Hosting projects on GitHub

12. Performance Optimization

- **Optimizing JavaScript**
 - Minification and bundling
 - Lazy loading
 - Debouncing and throttling
- **Web Performance APIs**
 - Timing APIs
 - Analyzing performance in DevTools

13. Advanced Topics (Optional)

- **TypeScript**
 - Introduction and usage with JavaScript
- **Node.js**
 - Setting up a backend
 - Creating REST APIs
- **WebAssembly**
 - Introduction and use cases

Vue.js (Progressive Web App-Javascript+Vue.js)

1. Prerequisites

Before starting Vue.js, ensure you have a good understanding of:

1. **JavaScript Fundamentals:**
 - ES6+ features: `let/const`, arrow functions, destructuring, template literals, modules, and promises.
 - DOM manipulation and event handling.
2. **HTML and CSS:**
 - Semantic HTML structure.
 - CSS basics, including Flexbox and Grid.
3. **Optional (React or Angular):**
 - Prior knowledge of a component-based framework can help but is not mandatory.

2. Introduction to Vue.js

- **What is Vue.js?**
 - Overview of Vue.js and its advantages.
 - Comparison with React and Angular.
 - Key concepts: reactivity, components, and single-file components (SFCs).
- **Setting Up Vue.js**
 - Installing Vue CLI.
 - Creating a Vue.js project.
 - Folder structure of a Vue.js application.

3. Vue Basics

- **Template Syntax**
 - Interpolations and directives (`v-bind`, `v-if`, `v-for`, `v-on`, `v-model`).
 - Using expressions and filters in templates.
- **Data and Methods**
 - Declaring reactive data properties.
 - Using methods for event handling.
- **Computed Properties and Watchers**
 - Creating computed properties for derived state.
 - Using watchers to respond to data changes.
- **Event Handling**
 - Handling DOM events with `v-on`.
 - Event modifiers (`.prevent`, `.stop`, `.self`).

4. Components

- **Introduction to Components**
 - Creating reusable components.
 - Passing data with `props` and emitting events.
- **Component Communication**
 - Parent-to-child and child-to-parent communication.
 - Using `provide` and `inject` for deep component communication.
- **Slots**
 - Understanding default slots and named slots.

- Scoped slots for dynamic content.

5. Directives and Forms

- **Built-in Directives**
 - Exploring `v-if`, `v-else`, `v-show`, `v-for`, and more.
 - Two-way binding with `v-model`.
- **Form Handling**
 - Working with input fields, checkboxes, radio buttons, and dropdowns.
 - Handling form submissions and validations.

6. Vue Router

- **Setting Up Vue Router**
 - Installing and configuring Vue Router.
 - Defining routes and dynamic route parameters.
- **Navigation**
 - Using `<router-link>` and `<router-view>`.
 - Programmatic navigation.
- **Nested Routes and Guards**
 - Creating nested routes and route guards for authentication.
- **Lazy Loading**
 - Implementing route-based code splitting.

7. Vuex for State Management

- **Introduction to Vuex**
 - What is Vuex and why use it?
 - Setting up Vuex in a Vue project.
- **Core Concepts**
 - State, Getters, Mutations, and Actions.
 - Managing centralized state with modules.
- **Advanced Vuex**
 - Using plugins and strict mode.

8. Working with APIs

- **Fetching Data**

- Making API calls with `axios` or the Fetch API.
- Handling loading and error states.
- **Displaying Data**
 - Rendering data with lists and handling pagination.
- **Real-Time Features**
 - Integrating WebSockets or Firebase for real-time updates.

9. Animations

- **Basic Animations**
 - Using transition and animation classes.
 - Animating elements with `v-if` and `v-show`.
- **Advanced Animations**
 - Using the `<transition>` and `<transition-group>` components.
 - Animating list items and custom transitions.

10. Testing

- **Unit Testing**
 - Writing unit tests for components with Jest.
- **Integration Testing**
 - Testing component interaction with Vue Test Utils.
- **E2E Testing**
 - End-to-end testing with Cypress.

11. Performance Optimization

- **Optimizing Vue Apps**
 - Lazy loading components and routes.
 - Using Vue's built-in optimization features (e.g., `v-once`, `v-bind:key`).
- **Profiling**
 - Profiling Vue apps with browser dev tools.

12. Deployment

- **Building for Production**

- Optimizing and building the app with `vue-cli-service build`.
- **Hosting**
 - Deploying Vue apps to platforms like Netlify, Vercel, or Firebase.

13. Advanced Topics

- **Composition API**
 - Understanding `setup()`, `reactive()`, and `ref()`.
 - Using the Composition API for complex logic.
- **TypeScript with Vue.js**
 - Adding TypeScript to a Vue project.
 - Defining types for props, data, and components.
- **Custom Directives**
 - Creating reusable custom directives.